UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL MENDOZA

TRABAJO PRÁCTICO INTEGRADOR

Food Store

Sistema de Gestión de Pedidos de Comida (Consola)

Programación Orientada a Objetos en Python

Universidad	Universidad Tecnológica Nacional (UTN)
Facultad	Facultad Regional Mendoza
Carrera	Tecnicatura Universitaria en Programación a Distancia
Asignatura	Programación II
Alumnos	Palazzolo López, Lucas Rubén Dacrema, Maximiliano
Profesorado	Tec. Rigoni Guerra, Cinthia Tec. Enferrel, Ariel Edgardo
Fecha de entrega	09 / 11 / 2026
Ciclo lectivo	2026

Mendoza, Argentina · 2026
Tecnicatura Universitaria en Programación a Distancia

Índice general

1. Introducción y alcance **6**

1.1 Contexto y propósito . . . 6
1.2 Objetivos de aprendizaje . . . 6
1.3 Información del proyecto . . . 6
1.4 Cumplimiento de la consigna . . . 7

2. Marco teórico **8**

2.1 Python y type hints . . . 8
2.2 Clase abstracta . . . 8
2.3 Interfaz . . . 8
2.4 Herencia y polimorfismo . . . 8
2.5 Encapsulamiento . . . 8
2.6 Composición y agregación . . . 9
2.7 Excepciones . . . 9
2.8 Colecciones y enumerados . . . 9
2.9 Arquitectura en capas y separación de responsabilidades . . . 9
2.10 Baja lógica y validaciones . . . 9
2.11 Pruebas y análisis estático . . . 9

3. Decisiones técnicas y arquitectura **10**

3.1 Organización en capas y paquetes . . . 10
3.2 Modelo de clases implementado . . . 11
3.3 Decisiones de diseño del modelo . . . 13
3.4 Servicios y reglas de negocio . . . 14
3.5 Reglas de negocio y dónde se implementan . . . 14
3.6 Jerarquía de excepciones propias . . . 15
3.7 Ciclo de vida de un pedido . . . 16
3.8 Flujo de creación de un pedido . . . 16
3.9 Inicio de sesión y ciclo del programa . . . 18
3.10 Consola y polimorfismo de menús . . . 19
3.11 Fragmentos de código destacados . . . 20
3.12 Trazabilidad: historias de usuario → implementación . . . 22
3.13 Diagramas editables en draw.io . . . 23

4. Capturas de ejecución **24**

4.1 Datos de ejemplo cargados por main() . . . 24

4.2 Inicio de sesión (HU-ACC-01, HU-ACC-02) . . . 24

4.3 Error de validación al crear un producto (HU-PROD-02) . . . 25

4.4 Pedido cancelado por falta de stock (HU-PED-02) . . . 25

4.5 Estados del pedido y devolución de stock (HU-PED-03) . . . 25

4.6 Menú de cliente (HU-PED-05, HU-USR-05) . . . 26

4.7 Tres intentos fallidos (HU-ACC-01) . . . 26

5. Verificación y pruebas **27**

6. Dificultades y soluciones **28**

7. Conclusiones **28**

7.1 Aprendizajes . . . 29

7.2 Posibles mejoras . . . 29

8. Bibliografía y webgrafía **29**

9. Repositorio y video **30**

Anexo A. Guía de uso paso a paso **31**

A.1 Recorrido como administrador . . . 31

A.2 Recorrido como cliente . . . 31

Anexo B. Estructura completa de archivos **31**

Anexo C. Glosario **32**

Índice de figuras

Figura 1. Capas del sistema y sentido de las dependencias (dibujado en draw.io; editable en FoodStore_Diagramas.drawio). 10

Figura 2. Diagrama de clases (draw.io): jerarquía de Base, composición Pedido–DetallePedido y agregación Categoría–Producto. 11

Figura 3. Diagrama de clases de la implementación, fiel al UML provisto (vista ampliada). 12

Figura 4. Jerarquía de excepciones propias de Food Store. 15

Figura 5. Estados y transiciones válidas de un pedido. 16

Figura 6. Diagrama de secuencia de la creación de un pedido entre consola, servicios y entidades. 17

Figura 7. Diagrama de flujo de la creación de un pedido. 18

Figura 8. Diagrama de flujo del inicio de sesión, la elección del menú y el cierre de sesión. 19

Figura 9. Inicio de sesión: un intento fallido y uno correcto; se muestra rol y menú del administrador. 24

Figura 10. Error de validación: el precio negativo se informa y se vuelve al menú sin cerrar el programa. 25

Figura 11. Pedido cancelado por falta de stock: no se guarda el pedido ni se modifica el stock. 25

Figura 12. Transición de estado inválida (PENDIENTE → TERMINADO) y luego cancelación válida. 25

Figura 13. Menú de cliente: sus pedidos con detalle y sus datos (sin contraseña). 26

Figura 14. Tras tres intentos fallidos el programa informa y termina. 26

Figura 15. Salida de python pruebas.py (extracto y resumen final). 27

Índice de tablas

Tabla 1. Información general del proyecto 6
Tabla 2. Cumplimiento de la consigna y la definición de completado 7
Tabla 3. Responsabilidad de cada capa 10
Tabla 4. Clases del modelo, herencia, atributos y relaciones 12
Tabla 5. Enumerados del dominio 13
Tabla 6. Operaciones públicas de cada servicio 14
Tabla 7. Reglas de negocio mínimas y su implementación 14
Tabla 8. Cuándo se lanza cada excepción 15
Tabla 9. Permisos por rol 20
Tabla 10. Estructura de los menús 20
Tabla 11. Trazabilidad de las historias de usuario 23
Tabla 12. Datos de ejemplo que carga el programa 24
Tabla 13. Resultado de pruebas.py por área 27
Tabla 14. Principales dificultades y soluciones 28
Tabla 15. Enlaces y recursos de la entrega 30
Tabla 16. Archivos del proyecto y su responsabilidad 31
Tabla 17. Glosario 32

1. Introducción y alcance

1.1 Contexto y propósito

Este informe documenta el desarrollo de **Food Store**, un sistema de consola para gestionar un negocio de comidas: categorías, productos, usuarios y pedidos. Se presenta como Trabajo Práctico Integrador de la asignatura **Programación II** de la Tecnicatura Universitaria en Programación a Distancia de la Universidad Tecnológica Nacional, Facultad Regional Mendoza. El sistema se implementó en **Python 3.12 o superior** utilizando únicamente la biblioteca estándar y aplicando el paradigma de Programación Orientada a Objetos (POO) según el diagrama UML provisto por la cátedra.

El programa inicia con un **inicio de sesión** (mail y contraseña). Según el rol del usuario se muestra el menú de **administrador** (alta, baja, modificación y consulta de todo) o el de **cliente** (solo consulta). Toda la información vive en memoria, en colecciones, y se pierde al cerrar el programa, tal como lo pide la consigna.

El desarrollo se guió por el backlog de épicas e historias de usuario del anexo de la consigna (Épica 0: acceso; Épica 1: categorías; Épica 2: productos; Épica 3: usuarios; Épica 4: pedidos y detalles): cada historia se transformó en una opción concreta del menú y en la lógica de servicios y entidades que la sostiene. Además se incorporaron validaciones de entrada, excepciones propias y una separación clara de responsabilidades entre modelo, servicios y consola.

1.2 Objetivos de aprendizaje

Al finalizar el trabajo, el proyecto evidencia que se puede:

- Modelar clases y relaciones del UML: herencia, asociación 1..N y N..1, agregación y composición.
- Aplicar herencia desde una clase abstracta (`Base`) y sobrescribir `__str__` y `__eq__`.
- Proteger el estado de los objetos con encapsulamiento y validaciones en los setters.
- Usar colecciones para gestionar las entidades y una interfaz (`Calculable`) para estandarizar un comportamiento.
- Crear, lanzar y capturar excepciones propias; aplicar polimorfismo para que cada rol tenga su menú sin condicionales repartidos.
- Documentar el contrato de cada clase y método con type hints (verificados con `mypy --strict`).

1.3 Información del proyecto

Tabla 1. Información general del proyecto

Aspecto	Definición
Modalidad	Aplicación de consola con menús
Lenguaje	Python 3.12 o superior (probado en 3.13)
Almacenamiento	En memoria (colecciones); los datos se pierden al cerrar
Capas	Entidades (modelo) + Servicios (lógica) + Consola (menús)

Aspecto	Definición
Bibliotecas	Solo biblioteca estándar: abc, enum, datetime (más typing, math, getpass y collections.abc)
Tamaño	31 archivos Python, 2026 líneas (incluye pruebas y comentarios)
Verificación	106 comprobaciones automáticas, mypy --strict sin errores, prueba con entradas aleatorias

1.4 Cumplimiento de la consigna

La consigna fija un alcance, una arquitectura recomendada y una «definición de completado». La siguiente tabla indica, punto por punto, cómo se cumplió y dónde se puede comprobar.

Tabla 2. Cumplimiento de la consigna y la definición de completado

Requisito de la consigna	Estado	Cómo se cumple / evidencia
El programa corre sin errores con python main.py	✔	Probado en Python 3.13; carga datos iniciales y muestra el inicio de sesión.
Alta, baja, modificación y consulta de categorías, productos, usuarios y pedidos	✔	Submenús de administrador (secciones 3.10 y 4); 22 historias de usuario trazadas en la sección 3.12.
main.py no contiene reglas de negocio	✔	main() solo crea servicios, carga datos y ejecuta el ciclo sesión → menú.
Inicio de sesión como ADMIN y USUARIO; cada uno ve solo sus opciones	✔	Menu abstracta + MenuAdministrador / MenuCliente; crear_menu() es el único lugar donde se consulta el rol.
Ningún dato mal ingresado cierra el programa (salvo 3 intentos de login)	✔	Lectura validada + captura central de FoodStoreError y ValueError; 300 corridas con entradas aleatorias sin caídas.
Todos los atributos, parámetros y retornos tienen type hints	✔	mypy --strict: «Success: no issues found in 31 source files».
pruebas.py corre y muestra casos válidos y errores esperados	✔	106 comprobaciones: [OK] y [ERROR ESPERADO] por pantalla (sección 5).
Clase abstracta, interfaz, herencia, polimorfismo, encapsulamiento, composición/agregación, excepciones, colecciones	✔	Base, Calculable, Menu; Pedido–DetallePedido (composición); Categoria–Producto (agregación); FoodStoreError; listas en servicios.
Informe en PDF, README.md y video de 10 a 15 minutos	✔	Este informe; README.md en la raíz; guion de video incluido (el enlace va en la sección 9).

2. Marco teórico

2.1 Python y type hints

Python es un lenguaje interpretado, de tipado dinámico y multiparadigma, con una sintaxis legible que lo hace adecuado para la enseñanza de la POO. Los **type hints** (PEP 484) son anotaciones opcionales que indican el tipo esperado de atributos, parámetros y valores de retorno (por ejemplo `def precio(self) -> float`). No cambian la ejecución, pero documentan el contrato de cada elemento y permiten que herramientas como `mypy` detecten inconsistencias antes de ejecutar. En este proyecto todos los atributos, parámetros y retornos están anotados.

2.2 Clase abstracta

Una clase abstracta es una clase que no puede instanciarse directamente y que existe para ser heredada. Define atributos y comportamiento comunes y puede declarar métodos abstractos que las subclases están obligadas a implementar. En Python se construye con el módulo `abc` (`ABC` y `@abstractmethod`). En el proyecto, `Base` es abstracta: concentra `id`, `eliminado` y `creado_en`, y obliga a cada entidad a implementar `__str__`. La clase `Menu` también es abstracta, con el método `mostrar()`.

2.3 Interfaz

Una interfaz define un contrato de comportamiento sin atributos ni implementación. Python no tiene la palabra clave *interface*, por lo que se modela con una clase abstracta que solo contiene métodos abstractos. `Calculable` declara `calcular_total() -> None`; `Pedido` la implementa (relación de realización del UML) y es la única clase que la usa.

2.4 Herencia y polimorfismo

La **herencia** permite que una clase (subclase) reutilice y extienda atributos y métodos de otra (superclase). Aquí, `Categoria`, `Producto`, `Usuario`, `Pedido` y `DetallePedido` heredan de `Base`. El **polimorfismo** permite tratar objetos de distintas clases a través de una interfaz común, y que cada uno responda a su manera. Se aplica en dos lugares: la sobrescritura de `__str__` y de `eliminar()` (en `Pedido` la baja se propaga a los detalles), y sobre todo en los menús: el programa llama a `menu.mostrar()` sin saber si es `MenuAdministrador` o `MenuCliente`.

2.5 Encapsulamiento

El encapsulamiento oculta el estado interno de un objeto y expone solo lo necesario. En Python se usan atributos privados con doble guion bajo (`self.__precio`, que el intérprete renombra internamente) y `@property` para exponerlos. Los setters validan el dato antes de modificarlo (por ejemplo, un precio negativo lanza `PrecioInvalidoError`). La contraseña de `Usuario` es el caso extremo: no tiene getter y solo se puede comparar con `verificar_contrasenia()`.

2.6 Composición y agregación

Ambas son asociaciones «todo-parte». En la **composición** la parte no existe sin el todo: un `DetallePedido` nace y muere con su `Pedido`, y solo este lo crea. En la **agregación** la parte tiene vida propia: una `Categoria` agrupa `Productos`, pero estos son entidades independientes. Además hay asociaciones simples: un `Usuario` realiza muchos `Pedidos` (1..N) y cada detalle refiere a un `Producto` (N..1).

2.7 Excepciones

Las excepciones son el mecanismo de Python para señalar y manejar errores en tiempo de ejecución con `raise` y `try/except`. Definir excepciones propias, todas derivadas de `FoodStoreError`, permite distinguir errores de negocio (id inexistente, stock insuficiente, mail duplicado, transición inválida) de errores de programación, y que la consola los capture para informar al usuario sin cerrar el programa.

2.8 Colecciones y enumerados

Las colecciones (`list`, `tuple`, `dict`, `frozenset`) almacenan y organizan objetos. Cada servicio guarda en una lista las entidades de su tipo; las propiedades que exponen listas internas devuelven tuplas para que nadie las modifique desde afuera. Los **enumerados** (`enum.Enum`) definen conjuntos cerrados de valores: `Rol`, `Estado` y `FormaPago`.

2.9 Arquitectura en capas y separación de responsabilidades

Dividir un programa en capas con una responsabilidad clara reduce el acoplamiento y facilita probar cada parte por separado. La regla principal es que la dependencia va en un solo sentido: la interfaz (consola) usa a la lógica (servicios) y esta a los datos del dominio (entidades), pero nunca al revés. Así, el modelo y los servicios se pueden ejecutar y probar sin ningún menú, y la consola podría reemplazarse por otra interfaz sin modificar las reglas de negocio. Este criterio se relaciona con el principio de responsabilidad única: cada clase tiene un motivo para cambiar.

2.10 Baja lógica y validaciones

La **baja lógica** consiste en marcar un registro como eliminado (`eliminado = True`) en lugar de borrarlo físicamente. Conserva el historial (por ejemplo, los pedidos que incluyen un producto dado de baja siguen mostrándose) y mantiene la integridad entre objetos relacionados. Las **validaciones** se aplican en dos niveles: la consola verifica el *formato* de lo que se escribe (números, textos vacíos, opciones del menú) y las entidades y servicios verifican las *reglas de negocio* (precio no negativo, mail único, transiciones de estado válidas).

2.11 Pruebas y análisis estático

Las pruebas automáticas ejecutan el código con casos válidos e inválidos y comprueban que el resultado o la excepción sea la esperada. El análisis estático (`mypy`) revisa los type hints sin ejecutar el programa. Ambas técnicas son complementarias y se usaron en este trabajo (sección 5).

3. Decisiones técnicas y arquitectura

3.1 Organización en capas y paquetes

El proyecto se organiza en tres capas, cada una en su paquete, con dependencia en un solo sentido: **consola → servicios → entidades**. Los paquetes `enums` y `excepciones` son transversales.

Tabla 3. Responsabilidad de cada capa

Capa / paquete	Responsabilidad	No hace
entidades/	Guardan su estado, lo validan en los setters y se describen con __str__.	Usar input() o print(); conocer las colecciones.
servicios/	Tienen la colección de su entidad; buscan por id; aplican reglas que involucran varios objetos (mail único, nombre de categoría único, stock, estados).	Leer del teclado o imprimir.
consola/	Pide datos, arma los objetos a través de los servicios, captura excepciones y muestra mensajes.	Contener reglas de negocio.
main.py	Crea los servicios, carga los datos iniciales y arranca el ciclo de sesión/menú.	Reglas de negocio.

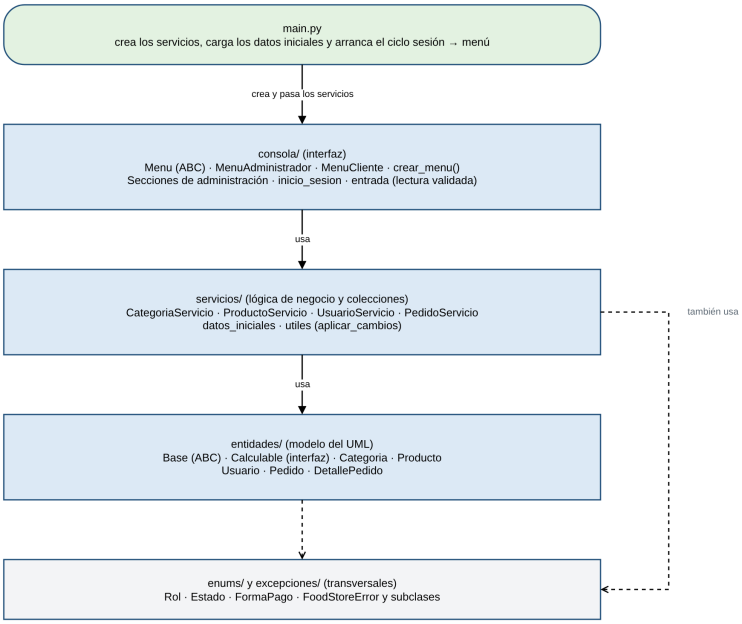


Figura 1. Capas del sistema y sentido de las dependencias (dibujado en draw.io; editable en FoodStore_Diagramas.drawio).

```
food_store/
├── main.py              # crea los servicios y arranca el menú principal
├── pruebas.py           # pruebas del modelo y servicios, sin menú
├── README.md            # ejecución, credenciales, enlaces y decisiones
├── entidades/           # Base, Calculable, Categoria, Producto, Usuario, Pedido, DetallePedido
├── enums/               # Rol, Estado, FormaPago
├── excepciones/         # FoodStoreError y subclases
├── servicios/           # CategoriaServicio, ProductoServicio, UsuarioServicio, PedidoServicio
└── consola/             # Menu, MenuAdministrador, MenuCliente, lectura validada
```

La separación se comprueba automáticamente en `pruebas.py`: un análisis del código verifica que `entidades/` y `servicios/` no llaman a `input()` ni `print()` y que no existen imports en sentido

contrario.

3.2 Modelo de clases implementado

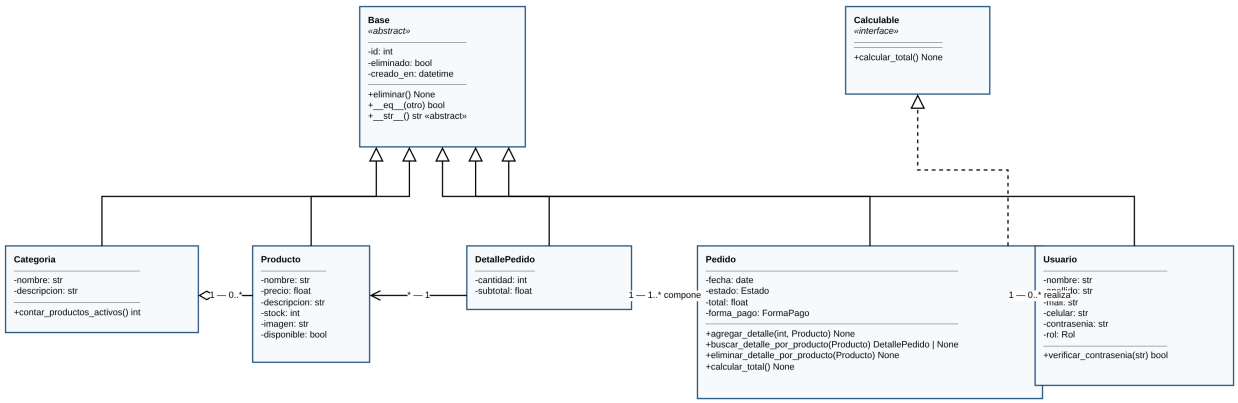


Figura 2. Diagrama de clases (draw.io): jerarquía de Base, composición Pedido–DetallePedido y agregación Categoría–Producto.

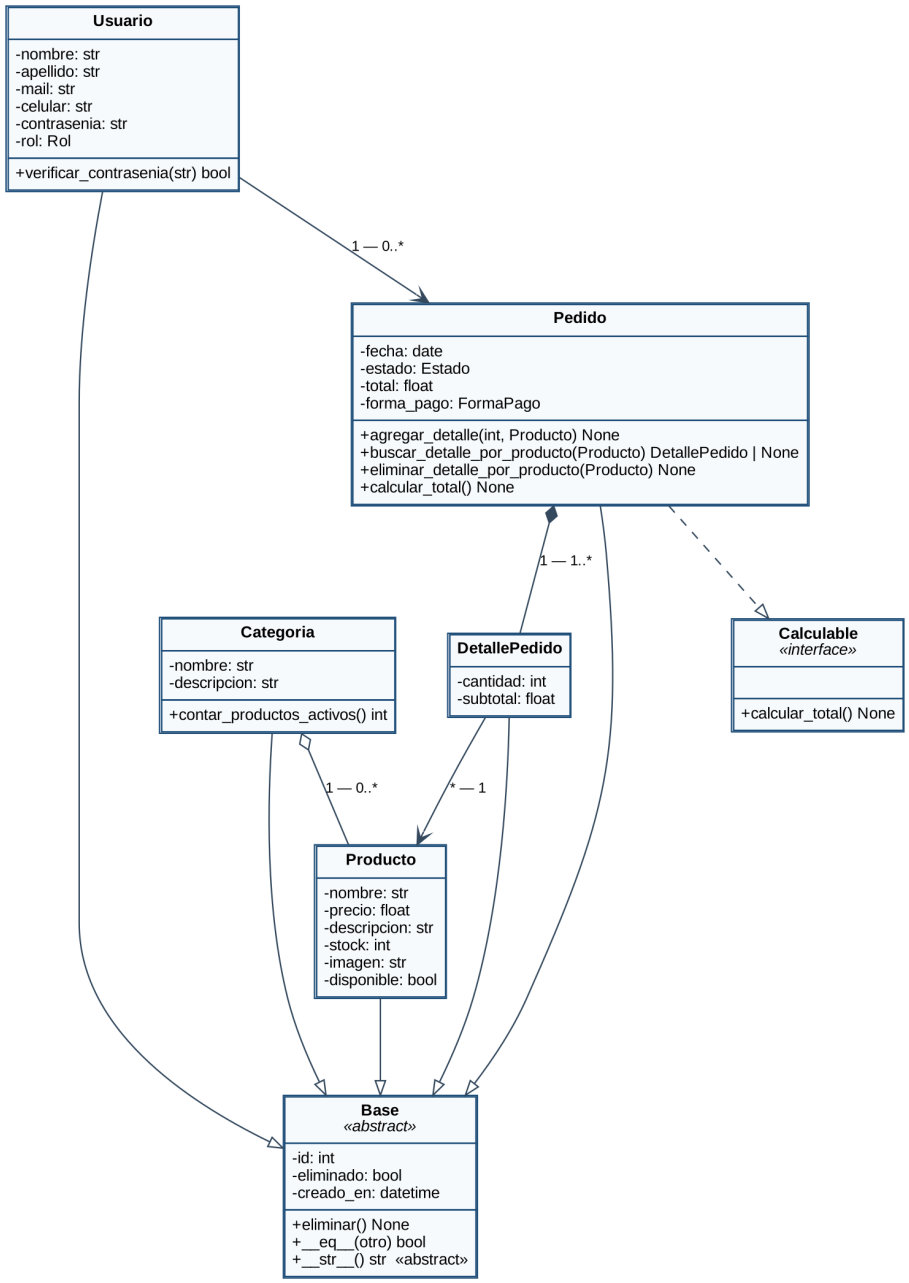


Figura 3. Diagrama de clases de la implementación, fiel al UML provisto (vista ampliada).

Tabla 4. Clases del modelo, herencia, atributos y relaciones

Clase	Hereda / implementa	Atributos propios (con tipo)	Relaciones
Base (abstracta)	ABC	id: int, eliminado: bool, creado_en: datetime	Superclase de todas las entidades
Categoria	Base	nombre: str, descripcion: str	Agregación 1..N con Producto
Producto	Base	nombre: str, precio: float, descripcion: str, stock: int, imagen: str, disponible: bool	N..1 con Categoria

Clase	Hereda / implementa	Atributos propios (con tipo)	Relaciones
Usuario	Base	nombre: str, apellido: str, mail: str (único), celular: str, contrasenia: str, rol: Rol	Asociación 1..N con Pedido
Pedido	Base, Calculable	fecha: date, estado: Estado, total: float, forma_pago: FormaPago	N..1 con Usuario; composición 1..N con DetallePedido
DetallePedido	Base	cantidad: int, subtotal: float	N..1 con Producto; vive dentro de un Pedido

Tabla 5. Enumerados del dominio

Enumerado	Valores	Uso
Rol	ADMIN, USUARIO	Define el menú y los permisos de cada usuario.
Estado	PENDIENTE, CONFIRMADO, TERMINADO, CANCELADO	Ciclo de vida del pedido; el propio enumerado conoce sus transiciones válidas (puede_pasar_a, es_final).
FormaPago	TARJETA, TRANSFERENCIA, EFECTIVO	Medio de pago del pedido; se elige por número en la consola.

3.3 Decisiones de diseño del modelo

- **Id por clase.** La consigna permite un contador compartido o uno por clase. Se eligió **uno por clase** (implementado con `__init_subclass__`) porque en la consola resulta más natural elegir «categoría 2» o «producto 5». Por eso `__eq__` compara *clase e id*, y `__hash__` se define de forma coherente (al definir `__eq__`, Python anula el hash heredado).
- **Baja lógica.** `Base.eliminar()` marca `eliminado = True` y el objeto permanece en la colección; los servicios filtran los eliminados. `Pedido` sobrescribe `eliminar()` para dar de baja también a sus detalles (composición).
- **Precio del detalle.** El subtotal se calcula con el precio del propio producto al agregarlo (*cantidad* × *precio*), redondeado a dos decimales. Un cambio posterior de precio no altera los pedidos ya armados; si el producto se agrega de nuevo al pedido, se suma la cantidad al detalle existente.
- **Producto «activo».** Se entiende como producto no eliminado: una categoría con productos activos no puede eliminarse. La relación Categoría–Producto se mantiene en ambos sentidos (el setter de `Producto.categoria` actualiza las dos puntas).
- **Estados.** Las transiciones válidas viven en el enumerado `Estado` (`puede_pasar_a()`, `es_final`), y el setter de `Pedido.estado` las valida. Un pedido TERMINADO o CANCELADO tampoco admite cambiar la forma de pago.
- **Contraseña.** Propiedad de solo escritura y método `verificar_contrasenia()`: nunca se imprime ni se lee. Se guarda en memoria sin cifrar por el alcance académico; un sistema real guardaría un hash con sal.
- **Mail.** Se valida formato básico (contiene «@» y «.») y se guarda en minúsculas; la unicidad se controla en `UsuarioServicio` recorriendo la colección de usuarios no eliminados.

- **Colecciones inmutables hacia afuera.** `Pedido.detalles` y `Categoria.productos` devuelven tuplas, por lo que no se pueden modificar desde fuera de la clase.

3.4 Servicios y reglas de negocio

Cada servicio tiene la lista de su entidad y métodos de alta, baja lógica, modificación y consulta. Los más relevantes:

- **Ediciones «todo o nada».** Al editar varios campos, la función `aplicar_cambios()` guarda los valores anteriores y los restaura si alguna validación falla, de modo que un error nunca deja un objeto a medias.
- **Creación de pedidos en tres pasos**: `iniciar_pedido()` (borrador fuera de la colección), `agregar_item()` (usa obligatoriamente `Pedido.agregar_detalle()`) y `confirmar_creacion()` (usa `calcular_total()`, revalida el stock, lo descuenta y recién entonces guarda el pedido). Si ocurre cualquier excepción antes de confirmar, el pedido no se agrega y no se modifica ningún stock.
- **Stock coherente.** Cancelar un pedido (o eliminar uno que no esté TERMINADO ni CANCELADO) devuelve el stock; eliminar uno TERMINADO no lo devuelve.
- **Autenticación.** `UsuarioServicio.autenticar()` lanza `CredencialesInvalidasError` con el mismo mensaje («Mail o contraseña incorrectos») si el mail no existe, el usuario está eliminado o la contraseña no coincide, para no revelar qué dato falló.
- **Sesión.** Un administrador no puede eliminarse a sí mismo ni quitarse su propio rol mientras tiene la sesión iniciada; así siempre queda al menos un administrador.

Tabla 6. Operaciones públicas de cada servicio

Servicio	Operaciones públicas
CategoriaServicio	listar, buscar_por_id, crear, editar, eliminar
ProductoServicio	listar, listar_disponibles, listar_por_categoria, buscar_por_id, crear, editar, eliminar
UsuarioServicio	listar, buscar_por_id, crear, editar, eliminar, autenticar
PedidoServicio	listar, listar_por_usuario, buscar_por_id, buscar_de_usuario, iniciar_pedido, agregar_item, confirmar_creacion, crear, actualizar, eliminar

3.5 Reglas de negocio y dónde se implementan

Tabla 7. Reglas de negocio mínimas y su implementación

Regla de negocio (consigna)	Implementación	Excepción
No se crea ni edita un producto con precio < 0 o stock < 0	validar_precio / validar_stock en los setters de Producto	PrecioInvalidoError, StockInvalidoError
No hay pedido sin usuario ni con usuario eliminado	Pedido.__init__ y PedidoServicio.iniciar_pedido	DatoInvalidoError, OperacionNoPermitidaError, EntidadNoEncontradaError
Cantidad del detalle > 0 y no mayor al stock	validar_cantidad y Pedido.agregar_detalle	CantidadInvalidaError, StockInsuficienteError

Regla de negocio (consigna)	Implementación	Excepción
Un pedido no repite producto: se suma la cantidad	Pedido.agregar_detalle + DetallePedido.aumentar_cantidad	—
El mail de usuario es único	UsuarioServicio (recorre la colección)	DuplicadoError
El nombre de categoría es único sin distinguir mayúsculas	CategoriaServicio (comparación en minúsculas)	DuplicadoError
No se elimina una categoría con productos activos	Categoria.contar_productos_activos + CategoriaServicio.eliminar	OperacionNoPermitidaError
Transiciones de estado válidas	Estado.puede_pasar_a + setter Pedido.estado + PedidoServicio.actualizar	OperacionNoPermitidaError
Cancelar/eliminar un pedido no terminado devuelve el stock	PedidoServicio.actualizar y eliminar	—
Toda baja es lógica	Base.eliminar (Pedido lo sobrescribe en cascada)	—
Solo un ADMIN crea, edita o elimina	MenuAdministrador (rechaza usuarios no ADMIN) + menú de cliente sin esas opciones	OperacionNoPermitidaError
Un usuario eliminado no inicia sesión	UsuarioServicio.autenticar recorre solo los no eliminados	CredencialesInvalidasError
Un administrador no se elimina a sí mismo	UsuarioServicio.eliminar(usuario_en_sesion)	OperacionNoPermitidaError

3.6 Jerarquía de excepciones propias

Todas las excepciones heredan de `FoodStoreError`. La consola captura esa clase base para mostrar mensajes de negocio y `ValueError` para las conversiones fallidas de `int()` y `float()`.

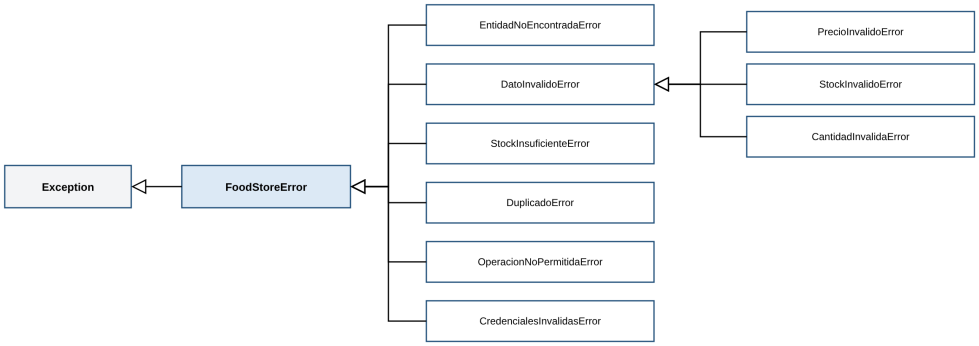


Figura 4. Jerarquía de excepciones propias de Food Store.

Tabla 8. Cuándo se lanza cada excepción

Excepción	Cuándo se lanza
EntidadNoEncontradaError	Id inexistente o eliminado; pedido de otro usuario consultado por un cliente; detalle inexistente en un pedido.
DatoInvalidoError	Texto vacío, tipo incorrecto, mail sin formato, rol o forma de pago inválidos, pedido sin detalles.

Excepción	Cuándo se lanza
PrecioInvalidoError / StockInvalidoError / CantidadInvalidaError	Precio negativo o no numérico; stock negativo o no entero; cantidad no entera o ≤ 0.
StockInsuficienteError	La cantidad pedida (sumando detalles repetidos) supera el stock del producto.
DuplicadoError	Mail de usuario o nombre de categoría repetido.
OperacionNoPermitidaError	Transición de estado inválida, categoría con productos activos, producto no disponible, pedido final modificado, administrador que se elimina o se cambia el rol.
CredencialesInvalidasError	Mail inexistente, usuario eliminado o contraseña incorrecta (mismo mensaje en los tres casos).

3.7 Ciclo de vida de un pedido

Un pedido nace PENDIENTE con la fecha del día. Solo puede avanzar por las transiciones válidas; TERMINADO y CANCELADO son finales. Al pasar a CANCELADO se devuelve el stock de todos sus productos.

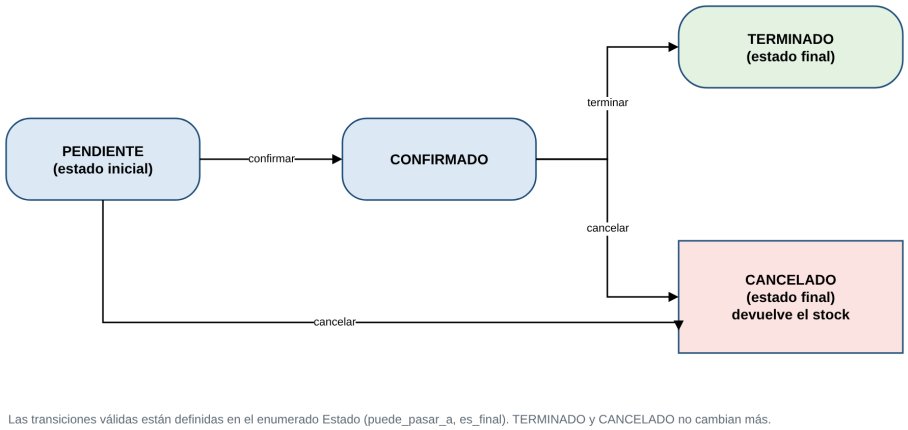


Figura 5. Estados y transiciones válidas de un pedido.

3.8 Flujo de creación de un pedido

La creación de un pedido es la operación más delicada porque involucra a varios objetos (usuario, productos, detalles y stock). Por eso se hace en tres pasos y el pedido recién entra a la colección al confirmarse: si cualquier validación falla antes, no se guarda nada y el stock queda intacto, como exige la historia HU-PED-02.

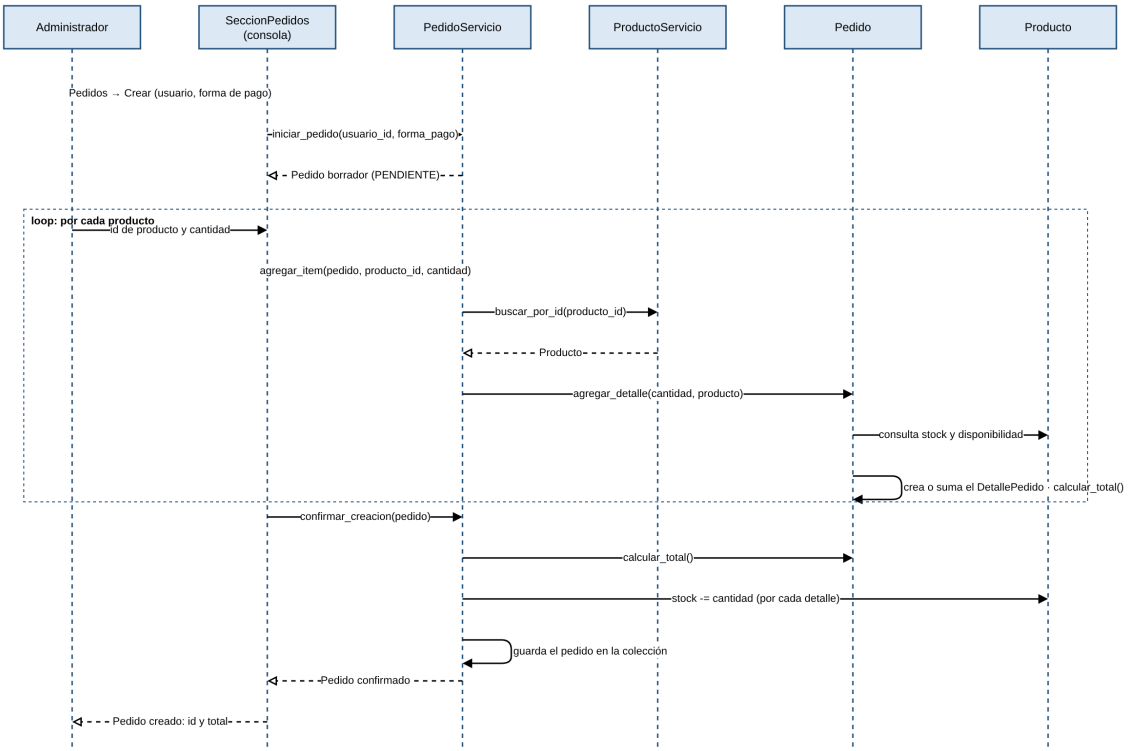


Figura 6. Diagrama de secuencia de la creación de un pedido entre consola, servicios y entidades.

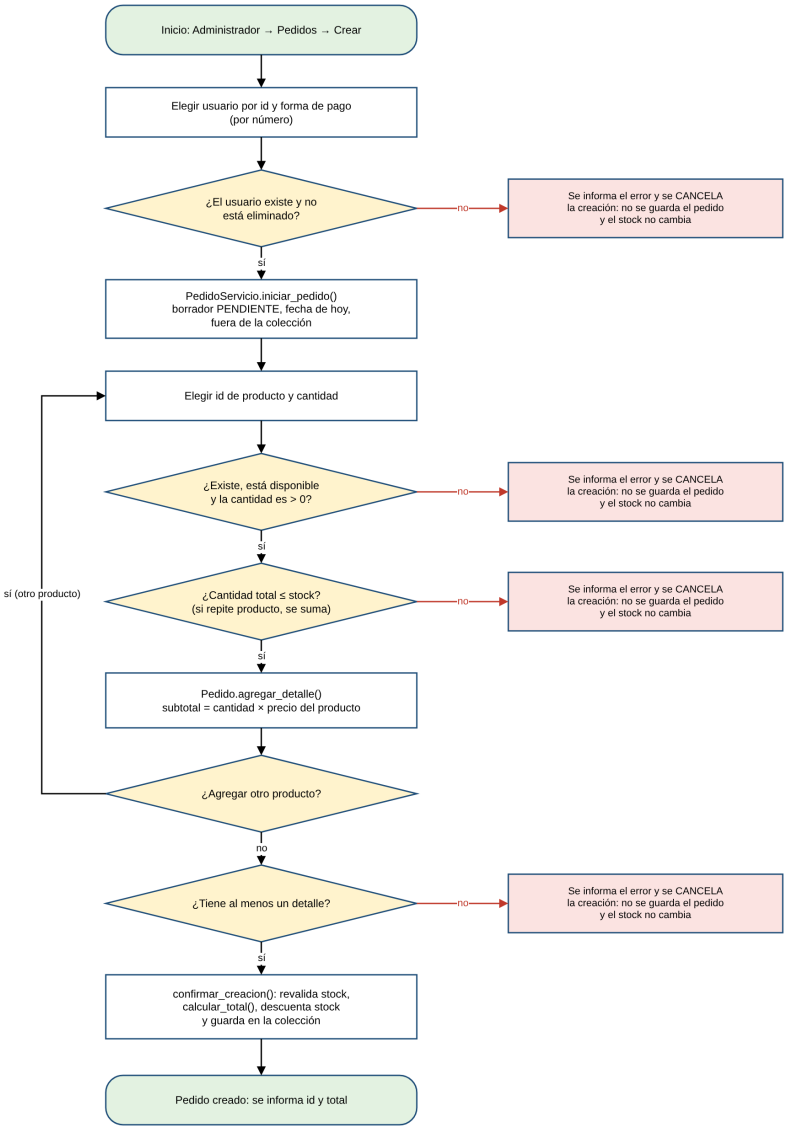


Figura 7. Diagrama de flujo de la creación de un pedido.

3.9 Inicio de sesión y ciclo del programa

Al ejecutar el programa se piden mail y contraseña con hasta tres intentos; al tercer error se informa y el programa termina. «Cerrar sesión» vuelve a la pantalla de inicio de sesión sin perder los datos cargados; «Salir» termina el programa.

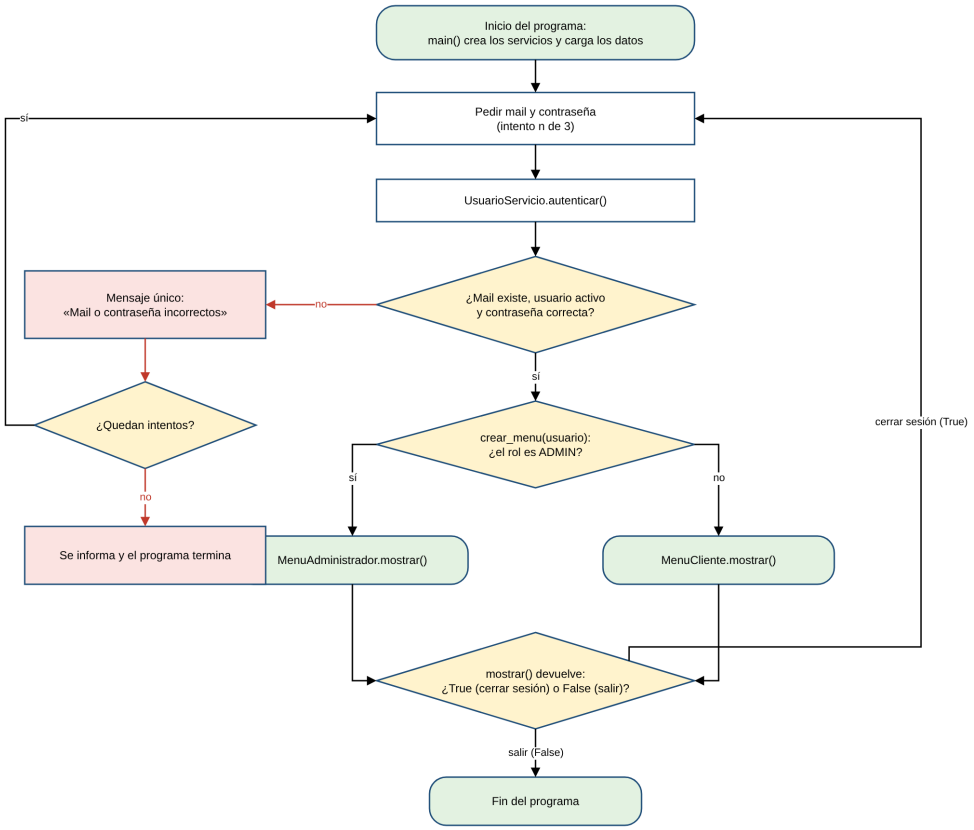


Figura 8. Diagrama de flujo del inicio de sesión, la elección del menú y el cierre de sesión.

3.10 Consola y polimorfismo de menús

`Menu` es una clase abstracta con el método `mostrar() -> bool` (True = cerrar sesión, False = salir). `MenuAdministrador` y `MenuCliente` lo implementan. El rol se consulta en un único lugar, `crear_menu()`; después el programa solo llama a `menu.mostrar()`. Agregar un rol nuevo significa agregar una subclase sin tocar las existentes. `MenuAdministrador` además se niega a construirse con un usuario que no sea ADMIN (defensa adicional).

```
def main() -> None:
    categorias = CategoriaServicio(); productos = ProductoServicio(categorias)
    usuarios = UsuarioServicio();     pedidos = PedidoServicio(usuarios, productos)
    cargar_datos_iniciales(categorias, productos, usuarios, pedidos)
    while True:
        usuario = iniciar_sesion(usuarios)       # la sesión es una variable local
        if usuario is None: break                # 3 intentos fallidos
        menu = crear_menu(usuario, categorias, productos, usuarios, pedidos)
        if not menu.mostrar(): break             # False = salir del programa
```

No hay variables globales: los servicios y la sesión se crean en `main()` y se pasan a los menús. La lectura por teclado (`consola/entrada.py`) valida formato (números, texto vacío, S/N, opciones fuera de rango) y repite la pregunta; las reglas de negocio se validan en entidades y servicios, y los errores se capturan en un único punto (`ejecutar()`) para informar y volver al menú. Los enumerados se eligen por número, nunca escribiendo el texto, y Enter conserva el valor anterior al editar. La contraseña se oculta con `getpass`.

Tabla 9. Permisos por rol

Sección	ADMIN	USUARIO (cliente)
Categorías	Listar, crear, editar, eliminar	Listar
Productos	Listar, crear, editar, eliminar, filtrar por categoría	Listar solo los disponibles y con stock
Usuarios	Listar, crear, editar, eliminar	Ver sus propios datos
Pedidos	Listar, crear, cambiar estado/forma de pago, eliminar, ver detalles	Ver solo sus propios pedidos y detalles

Tabla 10. Estructura de los menús

Menú	Opciones
Administrador	1 Categorías · 2 Productos · 3 Usuarios · 4 Pedidos · 9 Cerrar sesión · 0 Salir
Submenú de Categorías y Usuarios	1 Listar · 2 Crear · 3 Editar · 4 Eliminar · 0 Volver
Submenú de Productos	1 Listar · 2 Crear · 3 Editar · 4 Eliminar · 5 Listar por categoría · 0 Volver
Submenú de Pedidos	1 Listar todos · 2 Crear · 3 Cambiar estado y/o forma de pago · 4 Eliminar · 5 Listar por usuario · 6 Ver detalles · 0 Volver
Cliente	1 Ver categorías · 2 Ver productos disponibles · 3 Mis pedidos · 4 Mis datos · 9 Cerrar sesión · 0 Salir

3.11 Fragmentos de código destacados

Los siguientes fragmentos son copias exactas del código fuente entregado y muestran las decisiones centrales del diseño.

Base: id por clase, baja lógica e igualdad — `entidades/base.py`

```
class Base(ABC):
    """Atributos y comportamiento comunes: id, baja lógica y fecha de creación.

    Decisión de diseño: cada subclase tiene su propio contador de ids
    (Categoria 1, 2, 3...; Producto 1, 2, 3...), lo que hace más cómodo elegir
    por id desde la consola. Por eso `__eq__` compara clase e id.
    """

    _siguiente_id: ClassVar[int] = 1

    def __init_subclass__(cls, **kwargs: Any) -> None:
        """Cada subclase concreta arranca su propio contador en 1."""
        super().__init_subclass__(**kwargs)
        cls._siguiente_id = 1

    def __init__(self) -> None:
        clase = type(self)
        self.__id: int = clase._siguiente_id
        clase._siguiente_id += 1
        self.__eliminado: bool = False
        self.__creado_en: datetime = datetime.now()

    @property
    def id(self) -> int:
        """Identificador autogenerado; no se puede modificar."""
        return self.__id

    @property
```

```
    def eliminado(self) -> bool:
        """True si la entidad fue dada de baja (baja lógica)."""
        return self.__eliminado

    @property
    def creado_en(self) -> datetime:
        """Fecha y hora de creación, asignada en el constructor."""
        return self.__creado_en

    def eliminar(self) -> None:
        """Baja lógica: marca eliminado = True; el objeto sigue en la colección."""
        self.__eliminado = True

    def __eq__(self, otro: object) -> bool:
        """Dos entidades son iguales si son de la misma clase y tienen el mismo id."""
        if self is otro:
            return True
        if not isinstance(otro, Base):
            return NotImplemented
        return type(self) is type(otro) and self.id == otro.id

    def __hash__(self) -> int:
        """Coherente con __eq__ (al definir __eq__ Python anula el hash heredado)."""
        return hash((type(self).__name__, self.__id))

    @abstractmethod
    def __str__(self) -> str:
        """Cada entidad debe describirse en una línea útil para los listados."""
        ...
```

Pedido.agregar_detalle: validaciones y fusión de detalles — `entidades/pedido.py`

```
def agregar_detalle(self, cantidad: int, producto: Producto) -> None:
    """Agrega un detalle; si el producto ya estaba, suma la cantidad al existente.

    El precio sale del propio producto. Valida cantidad > 0, que el producto
    esté disponible y que la cantidad total no supere su stock.
    """
    self.__exigir_pendiente()
    if not isinstance(producto, Producto):
        raise DatoInvalidoError("El detalle debe referirse a un producto")
    if producto.eliminado or not producto.disponible:
        raise OperacionNoPermitidaError(f"El producto '{producto.nombre}' no está disponible")
    existente = self.buscar_detalle_por_producto(producto)
    cantidad_total = (existente.cantidad if existente else 0) + validar_cantidad(cantidad)
    if cantidad_total > producto.stock:
        raise StockInsuficienteError(
            f"Stock insuficiente para '{producto.nombre}': "
            f"solicitado {cantidad_total}, disponible {producto.stock}"
        )
    if existente is not None:
        existente.aumentar_cantidad(cantidad)
    else:
        self.__detalles.append(DetallePedido(cantidad, producto))
    self.calcular_total()
```

PedidoServicio.confirmar_creacion: stock y guardado «todo o nada» — `servicios/pedido_servicio.py`

```
def confirmar_creacion(self, pedido: Pedido) -> Pedido:
    """Calcula el total, descuenta el stock y recién entonces guarda el pedido."""
    if not pedido.detalles:
        raise DatoInvalidoError("El pedido debe tener al menos un detalle")
    for detalle in pedido.detalles:  # revalida todo antes de tocar nada
        if detalle.cantidad > detalle.producto.stock:
            raise StockInsuficienteError(
                f"Stock insuficiente para '{detalle.producto.nombre}': "
                f"solicitado {detalle.cantidad}, disponible {detalle.producto.stock}"
            )
    pedido.calcular_total()
    for detalle in pedido.detalles:
        detalle.producto.stock -= detalle.cantidad
```

```
    self.__pedidos.append(pedido)
    return pedido
```

aplicar_cambios: ediciones «todo o nada» — `servicios/utiles.py`

```
def aplicar_cambios(entidad: Base, cambios: dict[str, object]) -> None:
    """Asigna varios atributos "todo o nada": si uno falla, restaura los anteriores."""
    anteriores = {nombre: getattr(entidad, nombre) for nombre in cambios}
    try:
        for nombre, valor in cambios.items():
            setattr(entidad, nombre, valor)
    except FoodStoreError:
        for nombre, valor in anteriores.items():
            setattr(entidad, nombre, valor)
        raise
```

Estado: transiciones válidas — `enums/estado.py`

```
class Estado(Enum):
    """Estado de un pedido.

    Transiciones válidas:
        PENDIENTE  -> CONFIRMADO | CANCELADO
        CONFIRMADO -> TERMINADO  | CANCELADO
        TERMINADO y CANCELADO son estados finales (no cambian más).
    """

    PENDIENTE = "Pendiente"
    CONFIRMADO = "Confirmado"
    TERMINADO = "Terminado"
    CANCELADO = "Cancelado"

    def transiciones_validas(self) -> frozenset["Estado"]:
        """Devuelve los estados a los que se puede pasar desde este estado."""
        permitidas: dict[Estado, frozenset[Estado]] = {
            Estado.PENDIENTE: frozenset({Estado.CONFIRMADO, Estado.CANCELADO}),
            Estado.CONFIRMADO: frozenset({Estado.TERMINADO, Estado.CANCELADO}),
            Estado.TERMINADO: frozenset(),
            Estado.CANCELADO: frozenset(),
        }
        return permitidas[self]

    def puede_pasar_a(self, destino: "Estado") -> bool:
        """Indica si la transición desde este estado hacia `destino` es válida."""
        return destino in self.transiciones_validas()

    @property
    def es_final(self) -> bool:
        """True si el pedido ya no admite más cambios (TERMINADO o CANCELADO)."""
        return not self.transiciones_validas()
```

crear_menu: único lugar donde se consulta el rol — `consola/fabrica_menu.py`

```
def crear_menu(
    usuario: Usuario,
    categorias: CategoriaServicio,
    productos: ProductoServicio,
    usuarios: UsuarioServicio,
    pedidos: PedidoServicio,
) -> Menu:
    if usuario.rol == Rol.ADMIN:
        return MenuAdministrador(usuario, categorias, productos, usuarios, pedidos)
    return MenuCliente(usuario, categorias, productos, pedidos)
```

3.12 Trazabilidad: historias de usuario → implementación

Cada historia del backlog de la consigna se cumple con una opción de menú y la lógica que la sostiene. La tabla muestra dónde está cada una.

Tabla 11. Trazabilidad de las historias de usuario

Historia	Menú / pantalla	Lógica (servicio · entidad)
HU-ACC-01 Iniciar sesión	Pantalla de inicio de sesión	UsuarioServicio.autenticar · Usuario.verificar_contrasenia
HU-ACC-02 Menú según el rol	Menú principal	Menu, MenuAdministrador, MenuCliente, crear_menu
HU-ACC-03 Cerrar sesión	Opción 9	Menu.mostrar() devuelve True; ciclo de main()
HU-CAT-01 Listar categorías	Admin 1→1 · Cliente 1	CategoriaServicio.listar
HU-CAT-02 Crear categoría	Admin 1→2	CategoriaServicio.crear (nombre único)
HU-CAT-03 Editar categoría	Admin 1→3	CategoriaServicio.editar · aplicar_cambios
HU-CAT-04 Eliminar categoría	Admin 1→4	CategoriaServicio.eliminar · contar_productos_activos
HU-PROD-01 Listar productos	Admin 2→1/5 · Cliente 2	ProductoServicio.listar, listar_disponibles, listar_por_categoria
HU-PROD-02 Crear producto	Admin 2→2	ProductoServicio.crear · setters de Producto
HU-PROD-03 Editar producto	Admin 2→3	ProductoServicio.editar · aplicar_cambios
HU-PROD-04 Eliminar producto	Admin 2→4	ProductoServicio.eliminar · Base.eliminar
HU-USR-01 Listar usuarios	Admin 3→1	UsuarioServicio.listar (sin contraseña)
HU-USR-02 Crear usuario	Admin 3→2	UsuarioServicio.crear (mail único; se pide otro)
HU-USR-03 Editar usuario	Admin 3→3	UsuarioServicio.editar
HU-USR-04 Eliminar usuario	Admin 3→4	UsuarioServicio.eliminar(usuario_en_sesion)
HU-USR-05 Ver mis datos	Cliente 4	MenuCliente (sin contraseña)
HU-PED-01 Listar pedidos	Admin 4→1/5/6	PedidoServicio.listar, listar_por_usuario
HU-PED-02 Crear pedido con detalles	Admin 4→2	iniciar_pedido · agregar_item · confirmar_creacion · Pedido.agregar_detalle · calcular_total
HU-PED-03 Actualizar estado/forma de pago	Admin 4→3	PedidoServicio.actualizar · Estado.puede_pasar_a
HU-PED-04 Eliminar pedido	Admin 4→4	PedidoServicio.eliminar · Pedido.eliminar (cascada)
HU-PED-05 Consultar mis pedidos	Cliente 3	PedidoServicio.listar_por_usuario · buscar_de_usuario

3.13 Diagramas editables en draw.io

Todos los diagramas del informe (arquitectura, modelo de clases, flujo de creación de pedido, estados, inicio de sesión y jerarquía de excepciones) se entregan además en el archivo `FoodStore_Diagramas.drawio`, con una hoja por diagrama. Se abre con diagrams.net (draw.io) y permite modificar cajas, textos y flechas para exportarlos a PNG, SVG o PDF.

4. Capturas de ejecución

Las siguientes capturas corresponden a ejecuciones reales de `python main.py` con los datos de ejemplo que carga el programa. Cada una indica qué requisito o historia de usuario evidencia.

4.1 Datos de ejemplo cargados por main()

Tabla 12. Datos de ejemplo que carga el programa

Elemento	Datos
Usuarios	Ana Pérez (ADMIN) · admin@foodstore.com / admin123 — Juan Gómez (USUARIO) · juan@mail.com / juan123
Categorías	Pizzas · Hamburguesas · Bebidas · Postres
Productos	Pizza Muzzarella $9500 (stock 20) · Pizza Napolitana $11000 (15) · Hamburguesa Clásica $8000 (25) · Hamburguesa Veggie $8500 (10) · Gaseosa 500ml $2500 (50) · Cerveza Artesanal $4000 (30) · Tiramisú $5000 (2) · Flan Casero $4500 (8, no disponible)
Pedido inicial	De Juan Gómez, efectivo: 1 Pizza Muzzarella + 2 Gaseosas = $14500 (el stock queda en 19 y 48)

4.2 Inicio de sesión (HU-ACC-01, HU-ACC-02)

Se piden mail y contraseña (esta última oculta). El primer intento falla con el mensaje único «Mail o contraseña incorrectos»; el segundo ingresa y muestra el saludo con nombre y rol, y el menú de administrador.

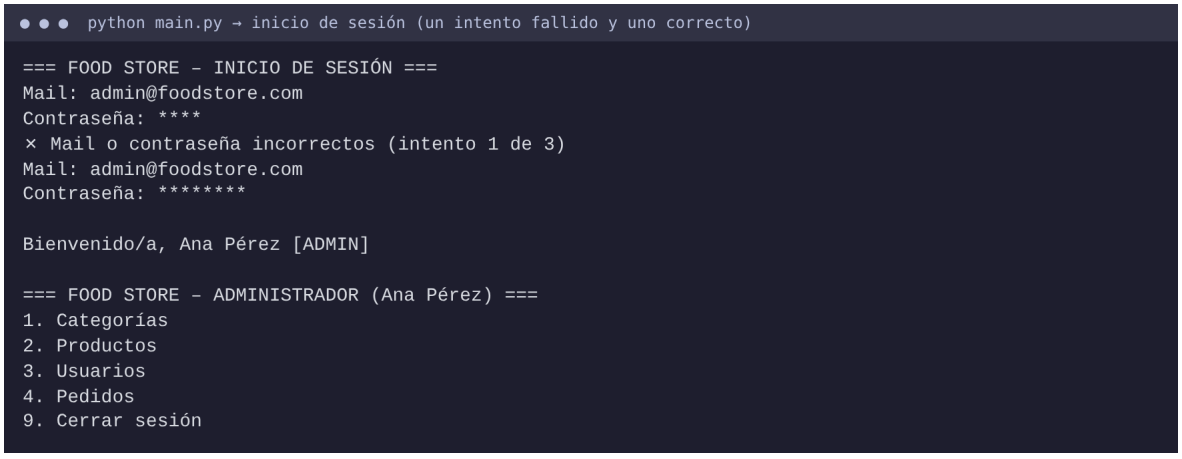


Figura 9. Inicio de sesión: un intento fallido y uno correcto; se muestra rol y menú del administrador.

4.3 Error de validación al crear un producto (HU-PROD-02)

Se carga un producto con precio -500. La entidad lanza PrecioInvalidoError, la consola muestra el mensaje y se vuelve al submenú: ningún error cierra el programa.

```
● ● ●  Administrador → Productos → Crear con precio negativo (validación)

Nombre: Café
Descripción (Enter para dejar vacía):
Precio: -500
Stock: 10
Imagen (Enter para dejar vacía):
¿Disponible? (S/N): S
  [1] Pizzas - Pizzas al horno de piedra
  [2] Hamburguesas - Hamburguesas de carne y vegetarianas
  [3] Bebidas - Gaseosas, aguas y cervezas
  [4] Postres - Dulces caseros
Id de la categoría: 1
× Error: El precio no puede ser negativo
```

Figura 10. Error de validación: el precio negativo se informa y se vuelve al menú sin cerrar el programa.

4.4 Pedido cancelado por falta de stock (HU-PED-02)

Se piden 5 unidades de Tiramisú, que tiene stock 2. Se informa «Stock insuficiente», se cancela la creación y se aclara que no se guardó el pedido ni se modificó el stock.

```
● ● ●  Administrador → Pedidos → Crear: falta de stock cancela la creación

Id del producto: 7
Cantidad: 5
× Stock insuficiente para 'Tiramisú': solicitado 5, disponible 2
Creación del pedido cancelada: no se guardó el pedido ni se modificó el stock.
```

Figura 11. Pedido cancelado por falta de stock: no se guarda el pedido ni se modifica el stock.

4.5 Estados del pedido y devolución de stock (HU-PED-03)

Pasar de PENDIENTE a TERMINADO se rechaza sin cambiar nada; PENDIENTE → CONFIRMADO → CANCELADO es válido y al cancelar se devuelve el stock.

```
● ● ●  Pedidos → cambio de estado inválido y luego cancelación

Seleccione número (Enter conserva):
✓ Pedido actualizado
  [1] Juan Gómez | PENDIENTE | EFECTIVO | total: $14500.00 | 30/10/2026
  [2] Juan Gómez | CONFIRMADO | TRANSFERENCIA | total: $18500.00 | 30/10/2026
Seleccione número (Enter conserva): 4
Seleccione número (Enter conserva):
✓ Pedido actualizado
  [1] Juan Gómez | PENDIENTE | EFECTIVO | total: $14500.00 | 30/10/2026
  [2] Juan Gómez | CANCELADO | TRANSFERENCIA | total: $18500.00 | 30/10/2026
```

Figura 12. Transición de estado inválida (PENDIENTE → TERMINADO) y luego cancelación válida.

4.6 Menú de cliente (HU-PED-05, HU-USR-05)

El cliente solo consulta: ve sus pedidos con detalle (producto, cantidad, subtotal y total) y sus datos personales, sin contraseña. No hay opciones que modifiquen datos.

```
● ● ●  Menú de cliente → Mis pedidos (con detalle) y Mis datos

Seleccione: 3
  [1] Juan Gómez | PENDIENTE | EFECTIVO | total: $14500.00 | 30/10/2026
Id del pedido para ver detalles (Enter para volver): 1
Pedido [1] Juan Gómez | PENDIENTE | EFECTIVO | total: $14500.00 | 30/10/2026
    - Pizza Muzzarella | cantidad: 1 | subtotal: $9500.00
    - Gaseosa 500ml | cantidad: 2 | subtotal: $5000.00
    TOTAL: $14500.00

=== FOOD STORE – CLIENTE (Juan Gómez) ===
1. Ver categorías
2. Ver productos disponibles
3. Mis pedidos
4. Mis datos
9. Cerrar sesión
0. Salir
Seleccione: 4
Nombre: Juan
Apellido: Gómez
Mail: juan@mail.com
Celular: 3815550002
Rol: USUARIO
```

Figura 13. Menú de cliente: sus pedidos con detalle y sus datos (sin contraseña).

4.7 Tres intentos fallidos (HU-ACC-01)

Al tercer intento incorrecto el programa informa «Se agotaron los 3 intentos» y termina: es la única situación en que el programa se cierra por datos mal ingresados.

```
● ● ●  Tres intentos fallidos: el programa informa y termina

=== FOOD STORE – INICIO DE SESIÓN ===
Mail: x@x.com
Contraseña: *
× Mail o contraseña incorrectos (intento 1 de 3)
Mail: admin@foodstore.com
Contraseña: *
× Mail o contraseña incorrectos (intento 2 de 3)
Mail: y
Contraseña: *
× Mail o contraseña incorrectos (intento 3 de 3)
Se agotaron los 3 intentos. El programa finaliza.
¡Hasta luego!
```

Figura 14. Tras tres intentos fallidos el programa informa y termina.

5. Verificación y pruebas

Además de la ejecución manual de todos los flujos, se realizaron tres verificaciones:

- `pruebas.py`: 106 comprobaciones sobre el modelo, los servicios, el polimorfismo de menús y la arquitectura, sin pasar por la consola. Muestra en pantalla cada caso válido *[OK]* y cada error esperado *[ERROR ESPERADO]* con su excepción y mensaje.
- **Análisis estático** con `mypy --strict` sobre los 31 archivos del proyecto: sin errores, lo que confirma que todos los atributos, parámetros y retornos tienen type hints correctos.
- **Entradas aleatorias** (300 corridas con 400 entradas cada una, incluyendo texto, números negativos, vacíos y opciones fuera de rango) sobre todos los menús: el programa nunca terminó por una excepción no controlada.

Tabla 13. Resultado de pruebas.py por área

Área probada	Casos válidos [OK]	Errores esperados	Total
MODELO: Base, Calculable y enumerados	7	2	9
MODELO: Categoria y Producto	5	7	12
MODELO: Usuario	4	4	8
MODELO: Pedido y DetallePedido (composición)	9	11	20
SERVICIOS: categorías	3	5	8
SERVICIOS: productos	5	5	10
SERVICIOS: usuarios y autenticación	5	6	11
SERVICIOS: pedidos	11	10	21
CONSOLA: polimorfismo de menús (sin interacción)	2	1	3
ARQUITECTURA: capas (consola -> servicios -> entidades)	4	0	4
TOTAL	55	51	106

```
● ● ●  python pruebas.py → casos válidos y errores esperados (extracto)

  [OK]              estado y forma de pago actualizados
  [OK]              cancelar devuelve el stock
  [OK]              eliminar pedido no terminado devuelve stock, marca pedido y detalles
  [OK]              el pedido eliminado no aparece en listados
  [ERROR ESPERADO]  Buscar pedido eliminado -> EntidadNoEncontradaError: No existe el pedido con id 8
  [OK]              eliminar pedido TERMINADO no devuelve stock
  [OK]              cada usuario ve solo sus pedidos
  [OK]              producto eliminado: el pedido existente se sigue mostrando
  [OK]              crear_menu(ADMIN) devuelve MenuAdministrador
  [OK]              crear_menu(USUARIO) devuelve MenuCliente
  [OK]              entidades no usan input() ni print()
  [OK]              servicios no usan input() ni print()
  [OK]              entidades no importan servicios ni consola
  [OK]              servicios no importan consola
   ...

Resumen: 106/106 pruebas correctas.
Todas las pruebas pasaron.
```

Figura 15. Salida de python pruebas.py (extracto y resumen final).

Qué cubren las pruebas
Precio y stock negativos, cantidades inválidas, stock insuficiente, fusión de detalles, transiciones de estado, devolución de stock, bajas lógicas, unicidad de mail y de nombre de categoría, credenciales inválidas, restricciones del administrador, polimorfismo de menús y separación de capas.

6. Dificultades y soluciones

Tabla 14. Principales dificultades y soluciones

Dificultad	Cómo se resolvió
Mantener __eq__ y __hash__ coherentes: al definir __eq__ Python deja el objeto sin hash y no se puede usar en conjuntos.	Se definió __hash__ en Base sobre (clase, id) y se verificó en las pruebas.
Ids por clase sin repetir código en cada subclase.	Un contador de clase (ClassVar) que cada subclase reinicia automáticamente con __init_subclass__.
Relación bidireccional Categoría–Producto con import circular entre módulos.	Categoria importa Producto solo para type hints (TYPE_CHECKING); el setter de Producto.categoria actualiza ambos lados.
Contraseña sin getter pero que se pueda asignar y verificar.	Propiedad de solo escritura (property(fset=...)) y verificar_contrasenia().
Pedido «todo o nada»: que un error al agregar un detalle no deje stock modificado.	El pedido se arma como borrador fuera de la colección; el stock se revalida y descuenta recién en confirmar_creacion().
Editar varios campos y que una validación tardía deje el objeto a medias.	aplicar_cambios() restaura los valores anteriores ante cualquier FoodStoreError; la contraseña (sin getter) se aplica al final.
Cambiar estado y forma de pago en una sola operación sin dejar cambios parciales.	Se valida primero la transición, luego se aplican los cambios; si algo no es válido no cambia nada.
Menú por rol sin if repartidos por el código.	Clase abstracta Menu con una subclase por rol; el rol se consulta solo en crear_menu().
Entradas inválidas por teclado (letras, vacíos, fuera de rango) sin cerrar el programa.	Funciones de lectura que repiten la pregunta, y captura centralizada de FoodStoreError y ValueError; verificado con entradas aleatorias.

Observación sobre los ids
Al ser autoincrementales, los ids de un pedido cuya creación se canceló se consumen, por lo que puede haber «saltos» en la numeración (comportamiento habitual en secuencias de bases de datos).

7. Conclusiones

El desarrollo permitió aplicar en un caso integrador los conceptos de la materia: una jerarquía de clases con clase abstracta e interfaz, encapsulamiento con validaciones, composición y agregación, colecciones, excepciones propias y polimorfismo en los menús. La separación en capas facilitó probar el modelo y los servicios sin la consola y mantener las reglas de negocio en un único lugar.

7.1 Aprendizajes

- Modelar primero (UML) y recién después programar reduce los cambios de diseño a mitad de camino.
- Las validaciones en los setters y las excepciones propias hacen que los errores sean predecibles y que la consola solo deba informarlos.
- Probar las capas por separado permitió detectar problemas antes de tener los menús listos.
- Las herramientas de análisis estático (mypy) obligan a pensar el contrato de cada método.

7.2 Posibles mejoras

- Persistencia en archivos o base de datos para no perder los datos al cerrar.
- Cifrado de contraseñas con hash y sal.
- Interfaz gráfica o web que reutilice las mismas capas de entidades y servicios.
- Paginación y búsqueda por texto en los listados, y reportes de ventas.

8. Bibliografía y webgrafía

- Python Software Foundation. *abc – Abstract Base Classes*. https://docs.python.org/3/library/abc.html
- Python Software Foundation. *enum – Support for enumerations*. https://docs.python.org/3/library/enum.html
- Python Software Foundation. *datetime – Basic date and time types*. https://docs.python.org/3/library/datetime.html
- Python Software Foundation. *typing – Support for type hints*. https://docs.python.org/3/library/typing.html
- Python Software Foundation. *getpass – Portable password input*. https://docs.python.org/3/library/getpass.html
- Python Software Foundation. *The Python Tutorial: Classes; Errors and Exceptions*. https://docs.python.org/3/tutorial/classes.html y https://docs.python.org/3/tutorial/errors.html
- Python Software Foundation. *The Python Language Reference: Data model* (__eq__, __hash__, __str__, __init_subclass__). https://docs.python.org/3/reference/datamodel.html
- van Rossum, G., Lehtosalo, J., Langa, Ł. *PEP 484 – Type Hints*. https://peps.python.org/pep-0484/
- van Rossum, G., Warsaw, B., Coghlan, N. *PEP 8 – Style Guide for Python Code*. https://peps.python.org/pep-0008/
- mypy project. *mypy documentation*. https://mypy.readthedocs.io/
- Fowler, M. (2003). *UML Distilled* (3.ª ed.). Addison-Wesley.
- diagrams.net. *draw.io – Diagramming software*. https://www.drawio.com/
- Cátedra de Programación II – UTN. *Trabajo Práctico Integrador – Food Store (Python)*. Consigna y material de la materia en el campus virtual.

9. Repositorio y video

Tabla 15. Enlaces y recursos de la entrega

Recurso	Enlace
Repositorio GitHub	[COMPLETAR: URL del repositorio]
Video demostración (10 a 15 min)	[COMPLETAR: URL del video con permisos públicos de visualización]
Informe en PDF	Incluido en el .zip de la entrega (TPI_FoodStore_Informe.pdf)
Diagramas editables	FoodStore_Diagramas.drawio (incluido en la entrega)

Cómo ejecutar
Python 3.12 o superior. Desde la carpeta food_store: `python main.py` para el sistema y `python pruebas.py` para las pruebas. Administrador inicial: `admin@foodstore.com` / `admin123`. Cliente de ejemplo: `juan@mail.com` / `juan123`.

Anexo A. Guía de uso paso a paso

A.1 Recorrido como administrador

- Ejecutar `python main.py` e ingresar `admin@foodstore.com` / `admin123`.
- Opción 1 (Categorías) → Crear: ingresar nombre y descripción; el sistema informa el id generado. Probar un nombre repetido en otra mayúscula para ver el error.
- Opción 2 (Productos) → Crear: ingresar nombre, descripción, precio, stock, imagen, disponibilidad (S/N) y el id de la categoría. Probar un precio negativo para ver la validación.
- Opción 3 (Usuarios) → Crear: ingresar nombre, apellido, celular, contraseña (oculta), rol (por número) y mail; si el mail existe, se pide otro.
- Opción 4 (Pedidos) → Crear: elegir usuario y forma de pago, agregar productos y cantidades, y confirmar. Probar 5 unidades de Tiramisú (stock 2) para ver la creación cancelada.
- Opción 4 → Cambiar estado y/o forma de pago: probar PENDIENTE → TERMINADO (inválido) y luego CONFIRMADO y CANCELADO (devuelve stock).
- Opción 9 para cerrar sesión o 0 para salir.

A.2 Recorrido como cliente

- Cerrar sesión e ingresar con `juan@mail.com` / `juan123`.
- Ver categorías y productos disponibles (no aparecen los no disponibles ni los sin stock).
- Mis pedidos: listar y elegir un id para ver el detalle; un id de otro usuario se informa como inexistente.
- Mis datos: muestra nombre, apellido, mail, celular y rol, nunca la contraseña.

Anexo B. Estructura completa de archivos

Tabla 16. Archivos del proyecto y su responsabilidad

Archivo	Líneas	Contenido
consola/__init__.py	1	Paquete
consola/entrada.py	121	Lectura validada por teclado
consola/fabrica_menu.py	20	crear_menu (elige el menú por rol)
consola/inicio_sesion.py	25	Inicio de sesión con 3 intentos
consola/menu.py	56	Menu (abstracta), ejecutar, submenú genérico
consola/menu_administrador.py	51	MenuAdministrador
consola/menu_cliente.py	61	MenuCliente
consola/secciones_admin.py	233	Submenús de categorías, productos, usuarios y pedidos
entidades/__init__.py	19	Exporta las entidades
entidades/base.py	64	Clase abstracta Base (id, baja lógica, __eq__, __hash__)

Archivo	Líneas	Contenido
entidades/calculable.py	15	Interfaz Calculable
entidades/categoria.py	57	Categoria
entidades/detalle_pedido.py	46	DetallePedido
entidades/pedido.py	154	Pedido (composición y Calculable)
entidades/producto.py	109	Producto
entidades/usuario.py	97	Usuario
entidades/validaciones.py	68	Validaciones reutilizables de los setters
enums/__init__.py	7	Exporta los enumerados
enums/estado.py	37	Estado y transiciones
enums/forma_pago.py	11	FormaPago
enums/rol.py	10	Rol
excepciones/__init__.py	41	FoodStoreError y subclases
main.py	30	Punto de entrada: crea servicios y ejecuta el ciclo sesión → menú
pruebas.py	333	Pruebas del modelo, servicios, menús y arquitectura
servicios/__init__.py	8	Exporta los servicios
servicios/categoria_servicio.py	54	CategoriaServicio
servicios/datos_iniciales.py	36	Administrador inicial y datos de ejemplo
servicios/pedido_servicio.py	100	PedidoServicio (stock y estados)
servicios/producto_servicio.py	70	ProductoServicio
servicios/usuario_servicio.py	76	UsuarioServicio y autenticación
servicios/utiles.py	16	aplicar_cambios (ediciones todo o nada)

Anexo C. Glosario

Tabla 17. Glosario

Término	Significado en este trabajo
ABC / clase abstracta	Clase que no se instancia y obliga a las subclases a implementar sus métodos abstractos (Base, Menu, Calculable).
Interfaz	Contrato de comportamiento sin atributos (Calculable).
Property / setter	Forma de exponer un atributo privado con lectura y escritura controladas; el setter valida.
Composición	Relación todo-parte donde la parte no existe sin el todo (Pedido–DetallePedido).
Agregación	Relación todo-parte donde la parte tiene vida propia (Categoria–Producto).
Polimorfismo	Tratar objetos distintos por una interfaz común (menu.mostrar()).

Término	Significado en este trabajo
Baja lógica	Marcar eliminado = True sin borrar el objeto de la colección.
Type hint	Anotación del tipo de un atributo, parámetro o retorno.
Servicio	Clase que guarda la colección de una entidad y aplica las reglas de negocio.
Borrador de pedido	Pedido PENDIENTE aún fuera de la colección, antes de confirmar la creación.
Transición de estado	Cambio permitido de un estado de pedido a otro.
mypy	Herramienta de análisis estático que verifica los type hints.